



طراحی جمع/تفریق کننده اعداد ممیز شناور

معماری کامپیوتر - گزارش کار تمرین عملی چهارم

استاد

دکتر سربازی آزاد

دانشگاه صنعتی شریف

بهار ۱۴۰۵

محمد مهدی مرادی - ۴۰۳۱۰۶۶۸۱

۱-مقدمه.....	۳
۲-معرفی زیر مدار ها(sub-curcuits).....	۳
۳-واحد کنترل (Control Unit - CU).....	۸
۴- بررسی مسیر داده (Datapath) به تفکیک وضعیت.....	۹
۵-نتیجه گیری.....	۱۶

۱. مقدمه

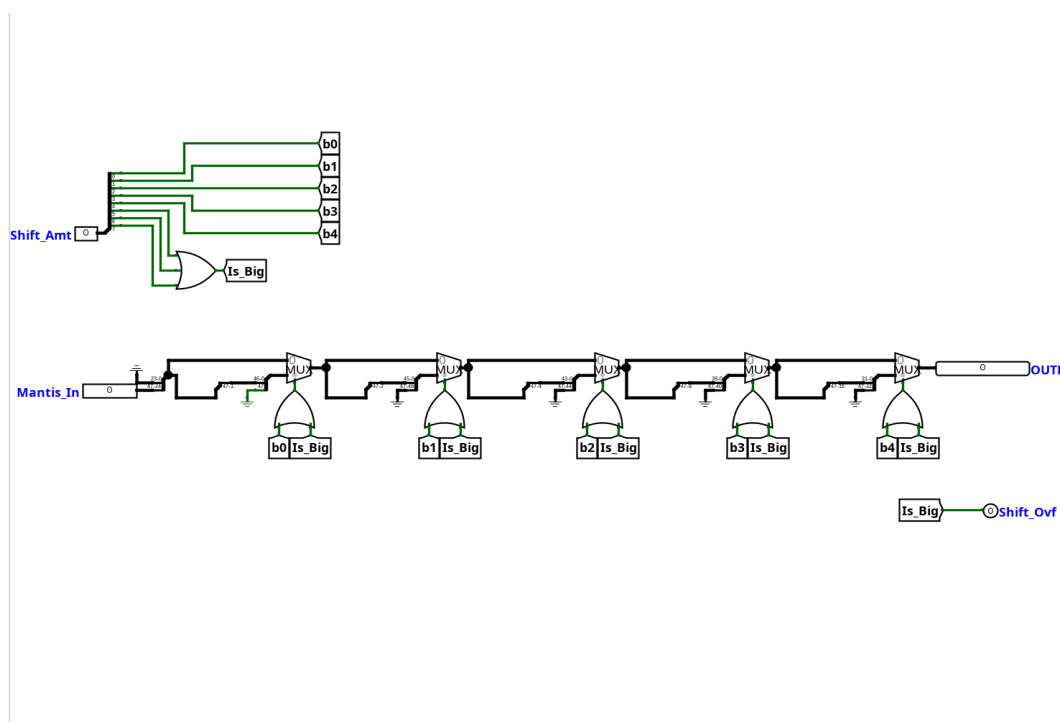
هدف از این پروژه، طراحی و پیاده‌سازی یک واحد جمع‌کننده و تفریق‌گر ممیز شناور (Floating-Point Adder/Subtractor) بر اساس استاندارد IEEE ۷۵۴-۳۲ bit در نرم‌افزار Logisim بوده است. این مدار قادر است دو عدد ۳۲ بیتی (A و B) را به همراه یک سیگنال کنترلی (Sub) دریافت کرده و حاصل جمع یا تفریق آن‌ها را با دقت بالا محاسبه کند. طراحی این پردازنده به دو بخش اصلی بخش کنترل (Control Unit) و مسیر داده (Datapath) تقسیم شده است. در این پروژه، چالش‌های پیچیده‌ای نظیر هم‌ترازی توان‌ها (Alignment)، نرمال‌سازی مجدد (Normalization)، پشتیبانی از اعداد بی‌نرمال (Denormal Numbers)، و گرد کردن دقیق (Rounding) با استفاده از بیت‌های Guard، Round و Sticky به طور کامل و موفقیت‌آمیز پیاده‌سازی شده است.

۲. معرفی زیرمدارها (Sub-circuits)

برای جلوگیری از شلوغی مدار اصلی و رعایت اصول طراحی ماژولار، عملگرهای پیچیده به صورت زیرمدار طراحی شده و در مسیر داده مورد استفاده قرار گرفتند:

۲-۱. شیفتر بشکهای (Barrel_Shifter)

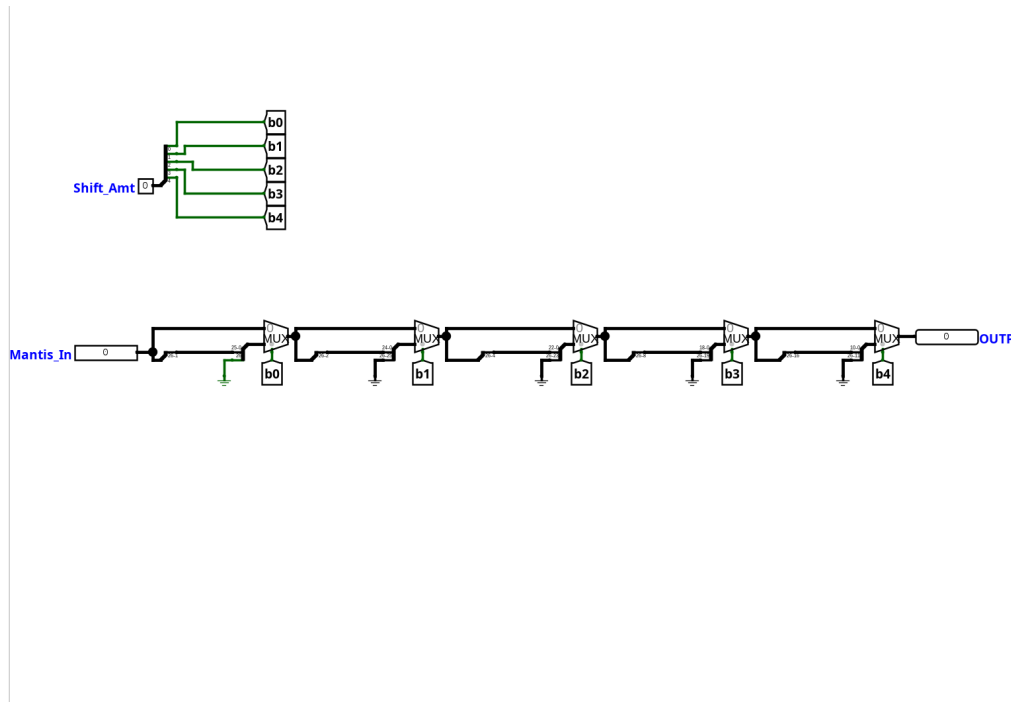
این زیرمدار وظیفه‌ی شیفتر به راست مانتیس را در مرحله‌ی هم‌ترازی (ALIGN) بر عهده دارد. اگر مقدار شیفتر از حد مجاز فراتر رود، سیگنال Is_Big یا Shift_Ovf فعال شده و شیفتر را روی عدد ۳۱ قفل می‌کند تا از بروز خطاهای محاسباتی جلوگیری شود. (شکل ۱)



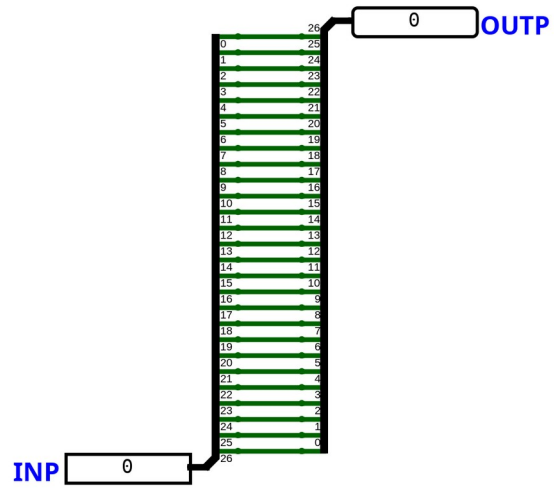
شکل ۱

۲-۲. شیفت‌ر چپ و معکوس‌کننده بیت (Shift_۲۷ و Bit_Reversal)

برای مرحله نرمال‌سازی (NORM)، نیاز به شیفت چپ داشتیم. این عمل با ترکیب دو زیرمدار Bit_Reversal (برای معکوس کردن ترتیب بیت‌ها) و استفاده از شیفت‌ر به راست، شبیه‌سازی شده است. این ساختار اجازه می‌دهد مانتیس پس از جمع/تفریق، به سرعت به سمت چپ شیفت خورده و نرمال شود. (شکل ۲ و ۳)



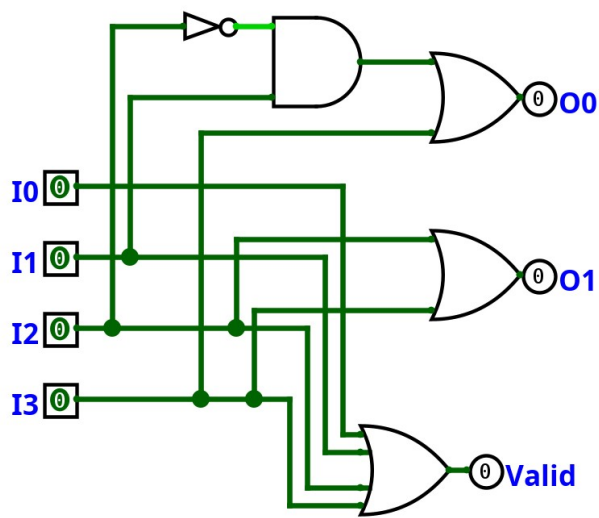
شکل ۲



شکل 3

۲-۳. انکودر اولویت‌دار (PE و Encoder_۴_۲)

برای پیدا کردن اولین بیت «۱» در مانتیس (Leading One) جهت نرمال‌سازی، از یک انکودر اولویت‌دار (Priority Encoder) سفارشی استفاده شده است. این ماژول با استفاده از بلوک‌های پایه‌ای Encoder_۴_۲ ساخته شده و می‌تواند دقیقاً مشخص کند که مانتیس نیاز به چند بیت شیفت چپ دارد. (شکل ۴ و ۵)



شکل 4



۳. واحد کنترل (Control Unit – CU)

واحد کنترل این پردازنده به صورت یک ماشین حالت متناهی (FSM) با ۸ وضعیت متمایز طراحی شده است. تغییر وضعیت‌ها با لبه‌ی بالارونده‌ی کلاک و بر اساس سیگنال‌های کنترلی تولید شده در Datapath (مانند Check_Done) انجام می‌شود. (شکل ۶)

وضعیت‌های این ماشین حالت عبارتند از:

۱. INIT: مقداردهی اولیه و ریست کردن رجیسترها.

۲. LOAD: بارگذاری مقادیر ورودی.

۳. CMP: مقایسه توان‌ها و محاسبه مقدار شیفت.

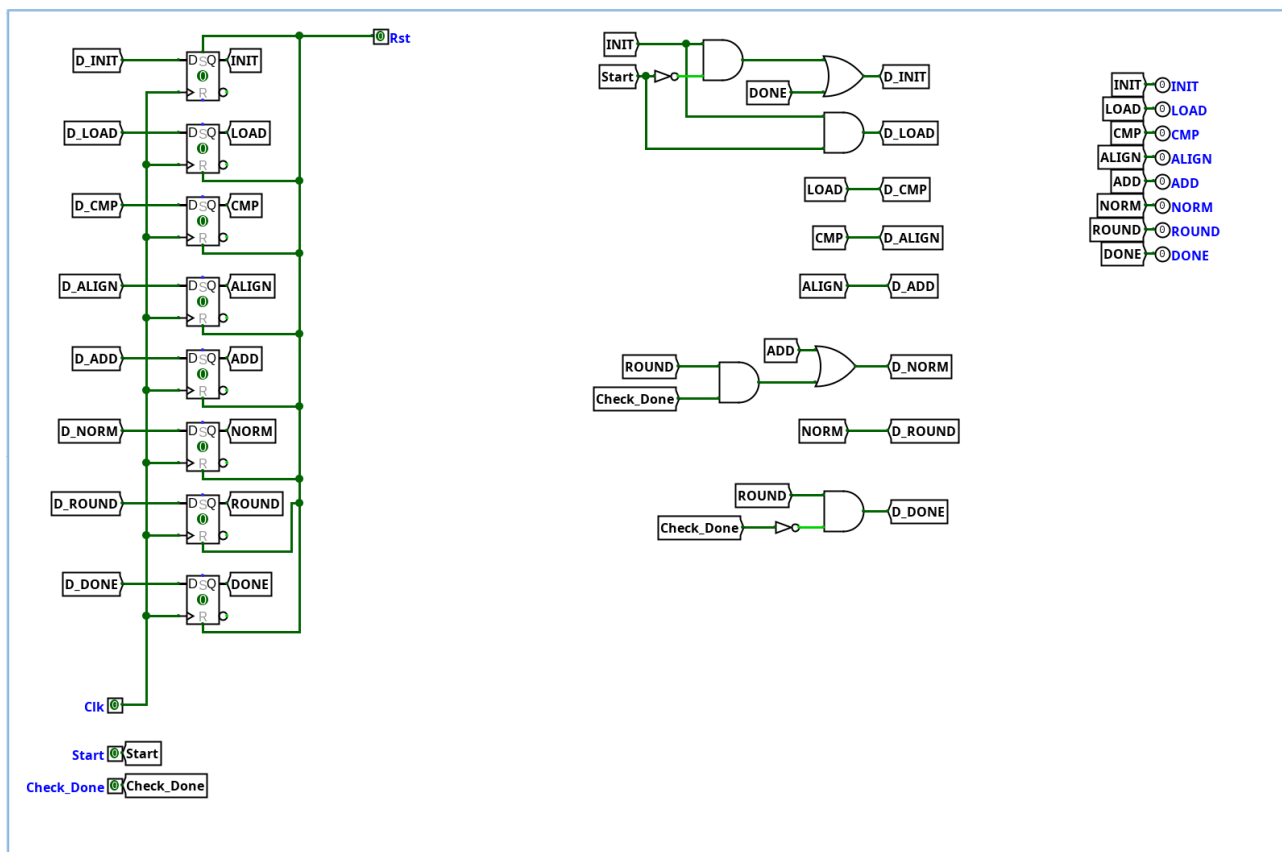
۴. ALIGN: هم‌ترازی مانتیس‌ها.

۵. ADD: انجام عملیات اصلی جمع/تفریق.

۶. NORM: نرمال‌سازی نتیجه.

۷. ROUND: گرد کردن حاصل.

۸. DONE: آماده‌سازی نتیجه نهایی.



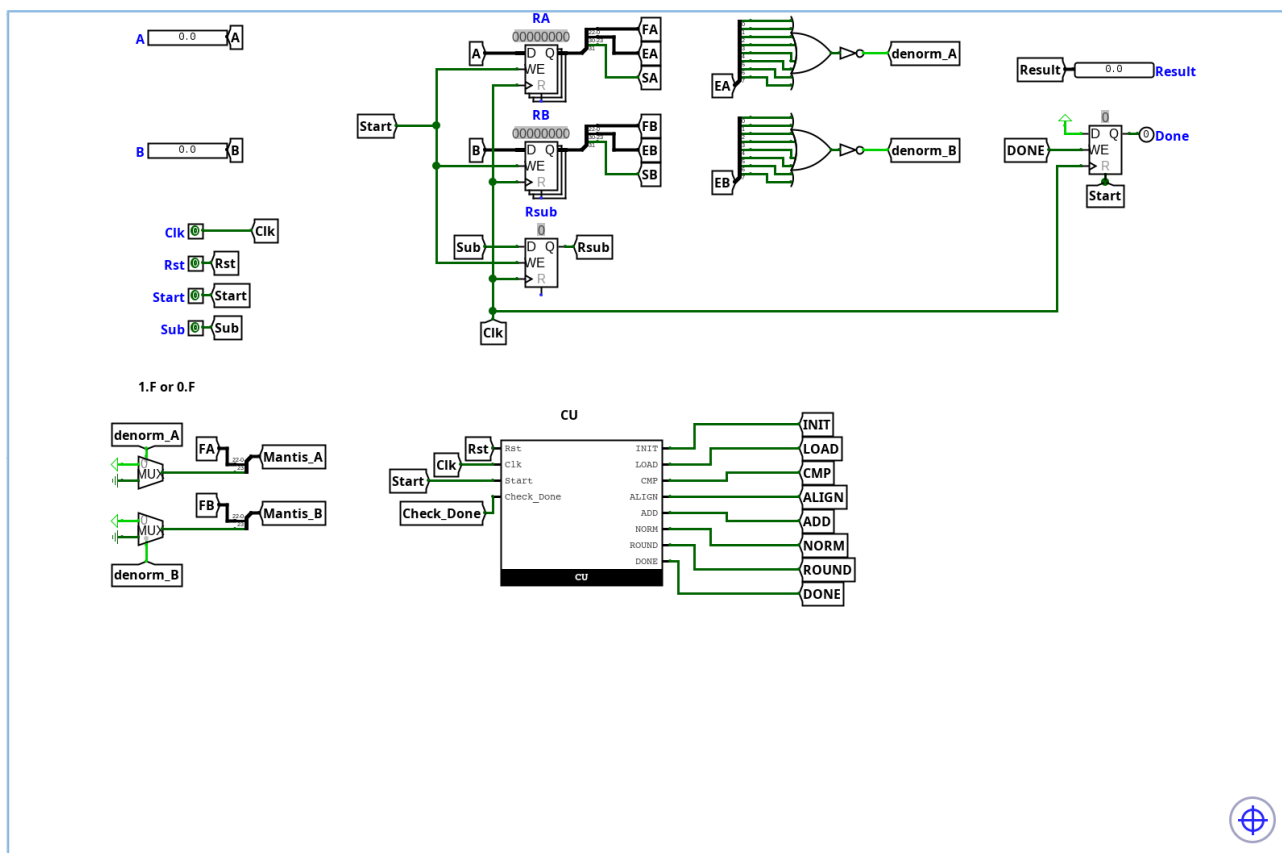
شکل 6

۴. بررسی مسیر داده (Datapath) به تفکیک وضعیت‌ها

مسیر داده‌ی اصلی (main) شامل تمام رجیسترها، مالتی‌پلکسرها و واحدهای حسابی است که تحت فرمانِ CU کار می‌کنند. در ادامه، مدار به تفکیک استیت‌های کاری تشریح می‌شود:

وضعیت بارگذاری (LOAD)

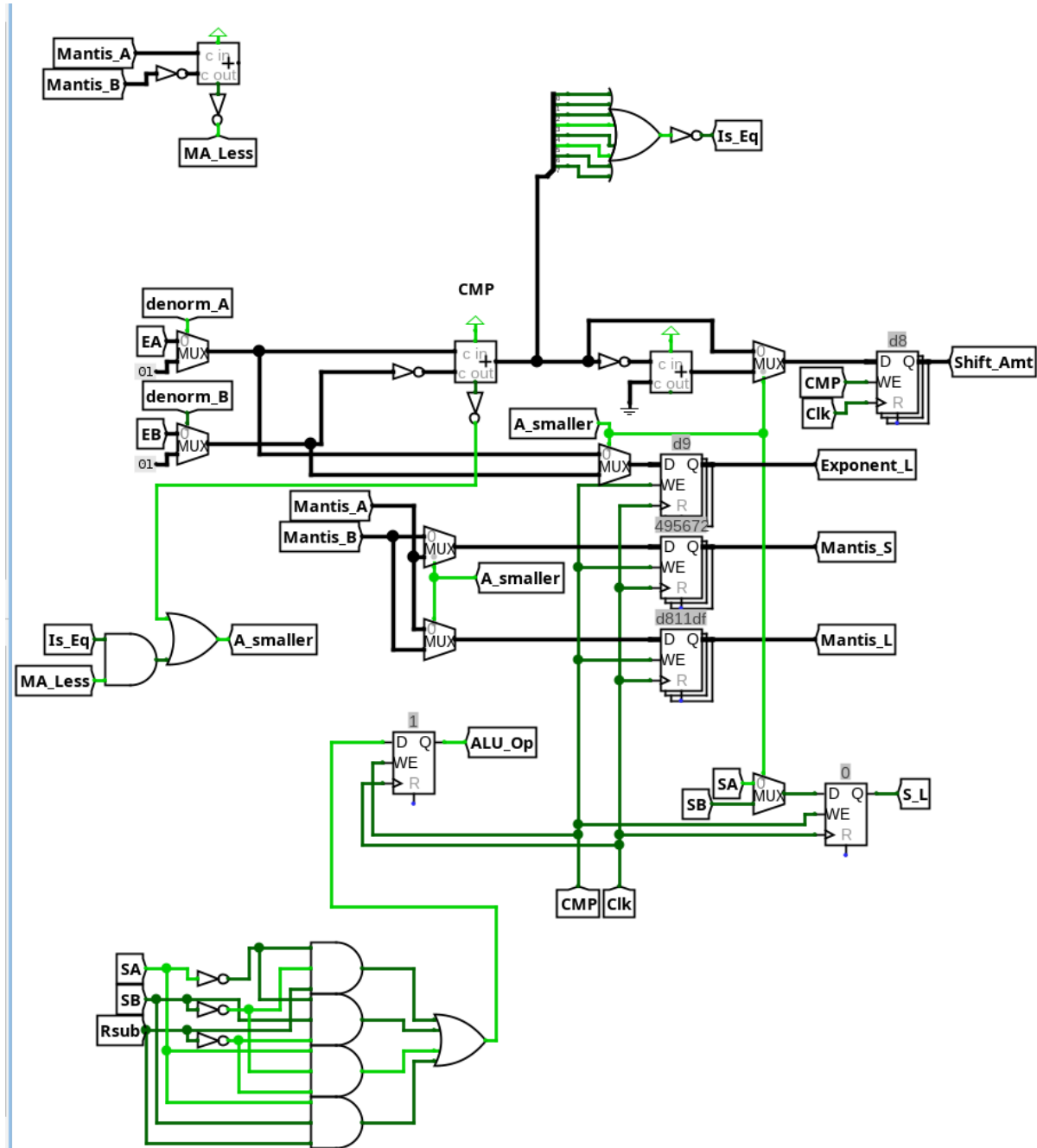
در این وضعیت، اعداد ۳۲ بیتی ورودی به سه بخش علامت (Sign)، توان (Exponent) و کسر (Fraction) تفکیک می‌شوند. بیت پنهان (Hidden Bit) با چک کردن صفر یا غیرصفر بودن توان، به کسر اضافه می‌شود تا مانتیس ۲۴ بیتی شکل بگیرد. همچنین سیگنال‌های denorm_A و denorm_B برای تشخیص اعداد بی‌نرمال در همین بخش تولید می‌شوند. (شکل ۷)



شکل 7

وضعیت مقایسه و محاسبه شیفت (CMP)

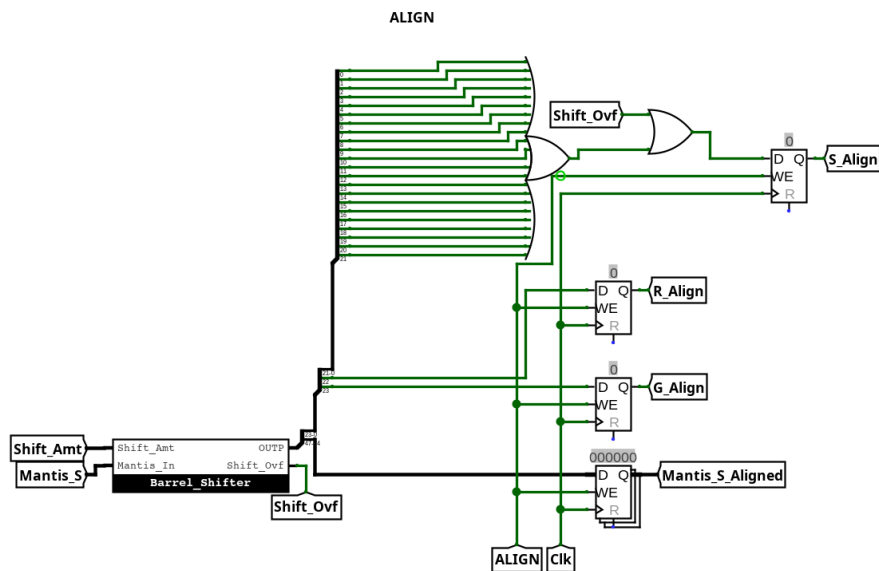
این وضعیت یکی از حساس‌ترین بخش‌های مدار است که وظیفه تعیین عدد کوچکتر و محاسبه اختلاف توان‌ها را بر عهده دارد. طراحی این بخش بدین شکل است: - برای تشخیص دقیق عدد کوچکتر ($A_{smaller}$)، تنها مقایسه توان‌ها کافی نیست. مدار به گونه‌ای طراحی شده است که در صورت برابر بودن توان‌ها (تشخیص داده شده توسط یک گیت NOR هشت ورودی با نام Is_Eq)، یک مدار تفریق ۲۴ بیتی وارد عمل می‌شود. این تفریق گرمانتیس‌ها را با منطق مکمل دو مقایسه کرده و سیگنال MA_Less را تولید می‌کند. - سیگنال نهایی $A_{smaller}$ از ترکیب منطقی این شرایط به دست می‌آید: (توان A کوچکتر باشد) یا (توان‌ها مساوی باشند و مانتیس A کوچکتر باشد). - در نهایت اختلاف توان‌ها محاسبه شده و به عنوان مقدار شیفت ($Shift_Amt$) ذخیره می‌شود و توان عدد بزرگتر به عنوان توان پایه ($Exponent_L$) به مرحله بعد می‌رود. توان عدد بزرگتر (SL) مانتیس عدد بزرگتر ($Mantis_L$) و مانتیس عدد کوچکتر ($Mantis_S$) را نیز در رجیسترها ذخیره می‌کنیم. (شکل ۸)



شکل 8

وضعیت هم‌ترازی (ALIGN)

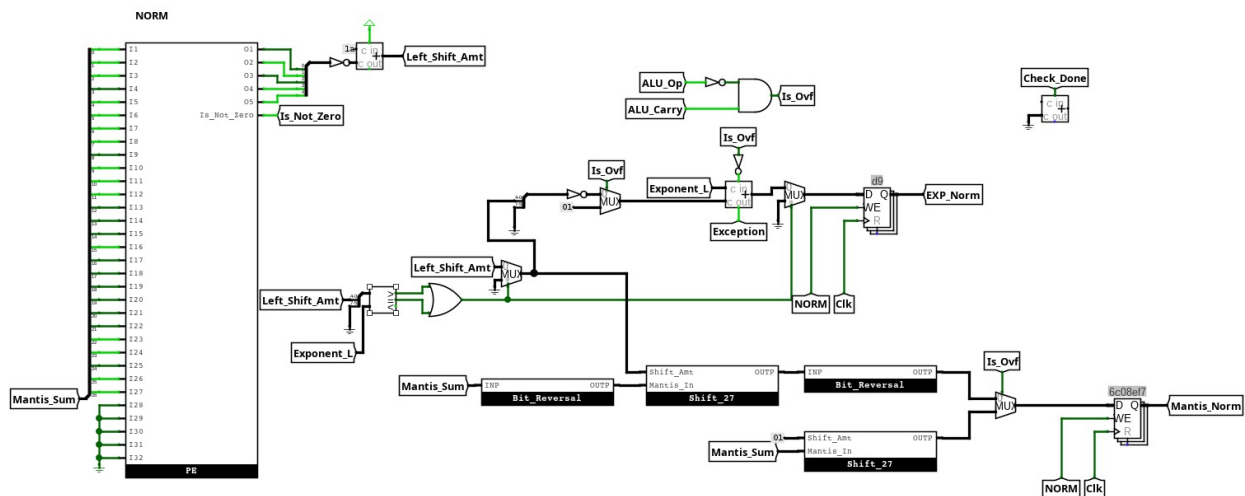
مانتیس عدد کوچکتر (Mantis_S) به اندازه‌ی Shift_Amt به راست شیفت داده می‌شود تا هم‌توان عدد بزرگتر شود. در این مرحله، بیت‌هایی که از سمت راست کسر خارج می‌شوند، برای تشکیل بیت‌های Guard، Round و Sticky جمع‌آوری می‌شوند. منطق Shift_Ovf در اینجا تضمین می‌کند که اگر مقدار شیفت بسیار بزرگ بود، بیت Sticky به درستی ۱ شود تا خطای گرد کردن رخ ندهد. (شکل ۹)



شکل 9

وضعیت محاسبه (ADD)

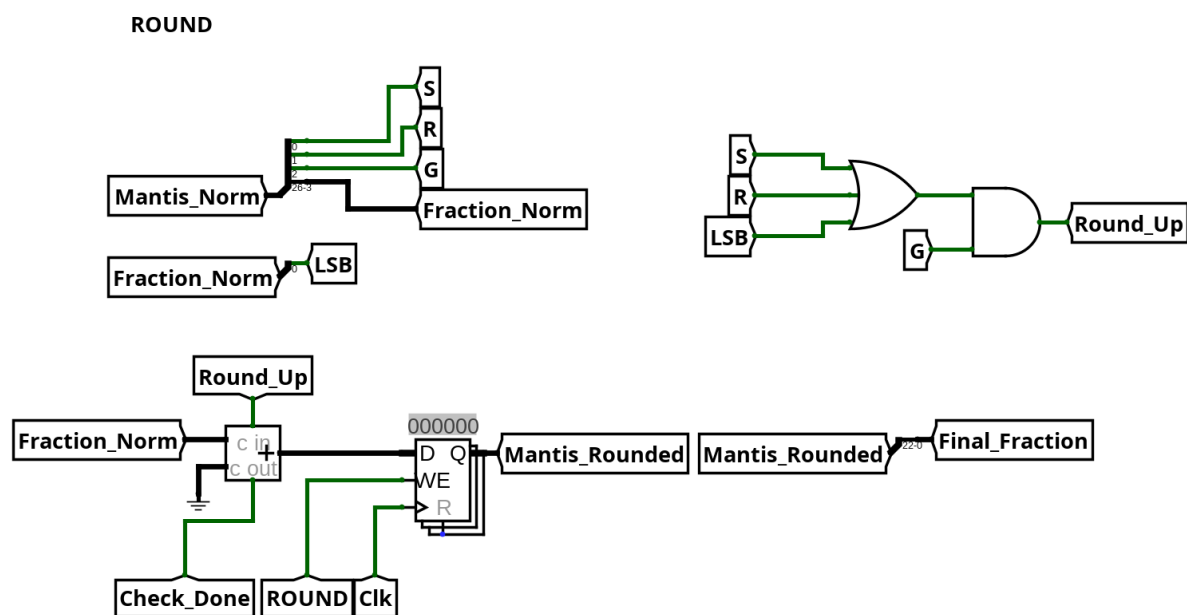
عملیات اصلی حسابی (ALU) در این بخش انجام می‌شود. با توجه به علامت اعداد ورودی و پایهی کنترلی Sub، مدار تصمیم می‌گیرد که آیا مانتیس‌ها باید با هم جمع شوند یا تفریق. حاصل این عملیات در رجیستر Mantis_Sum ذخیره شده و رقم نقلی خروجی (Carry) نیز برای استفاده در استیت بعدی در ALU_Carry قرار می‌گیرد. (شکل ۱۰)



شکل 11

وضعیت گرد کردن (ROUND)

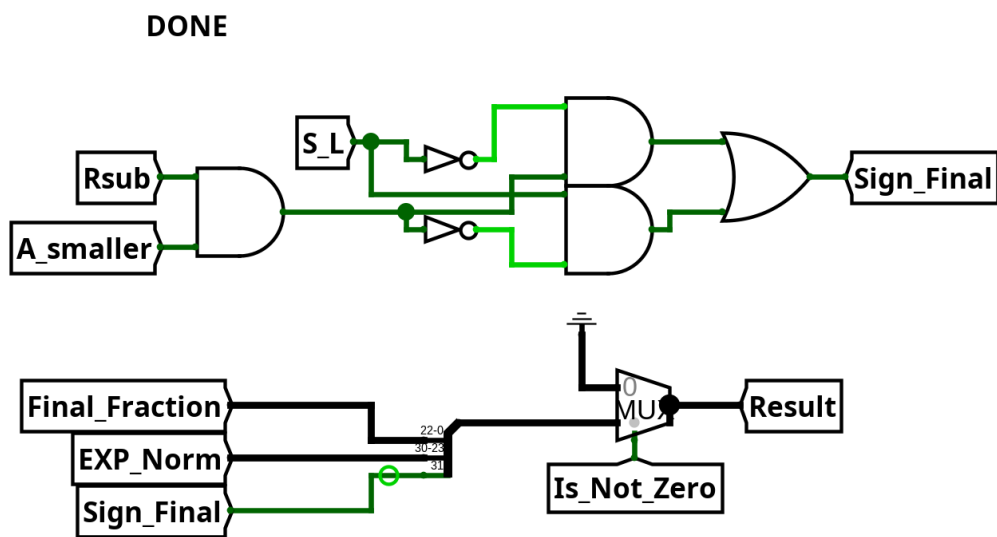
با استفاده از ۳ بیت پایانی (G, R, S) و کم‌ارزش‌ترین بیت کسر (LSB)، مدار تصمیم می‌گیرد که آیا نیاز به اضافه کردن یک واحد (Round Up) هست یا خیر. اگر این گرد کردن باعث سرریز مانتیس شود، سیگنال Check_Done فعال شده و دوباره به حالت NORM برمیگردیم تا جواب ما نرمال شود و اگر سیگنال Check_Done غیر فعال باشد به حالت DONE می‌رویم. (شکل ۱۲)



شکل 12

وضعیت اتمام (DONE)

در نهایت، بیت علامت نهایی (**Sign_Final**) بر اساس عملوندها تعیین شده، توان آپدیت شده (**EXP_Norm**) و مانتیس گرد شده (**Final_Fraction**) در کنار هم بسته‌بندی می‌شوند تا خروجی ۳۲ بیتی استاندارد (**Result**) را بسازند. همچنین اگر سیگنال **Is_Not_Zero** که از انکودر اولویت دار گرفته شده غیرفعال باشد یعنی خروجی نهایی باید عدد صفر شود که این مکانیزم را با یک مولتی پلکسر که خط انتخاب آن همین سیگنال است، پیاده کردیم. (شکل ۱۳)



شکل 13

۵. نتیجه‌گیری

در این پروژه تمامی ماژول‌های منطقی بدون استفاده از قطعات آماده‌ی سطح بالا، و با تکیه بر اصول طراحی دیجیتال پیاده‌سازی شدند. خروجی گرفته شده از تست بنچ را در شکل ۱۴ ببینید. (۱۰۰۰/۱۰۰۰)

```

[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "D Flip-Flop" in circuit "CU" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "D Flip-Flop" in circuit "CU" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "D Flip-Flop" in circuit "CU" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "D Flip-Flop" in circuit "CU" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "D Flip-Flop" in circuit "CU" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "D Flip-Flop" in circuit "CU" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "D Flip-Flop" in circuit "CU" has a gated clock connection!
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/HW4-403106681.circ.tmp/main/verilog/circuit/CU.v
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Clock component found with no connection, skipping: ''
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/HW4-403106681.circ.tmp/main/verilog/circuit/main.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/HW4-403106681.circ.tmp/main/verilog/topLevel/logisimTopLevelShell.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /home/mahdi/logisim_evolution_workspace/HW4-403106681.circ.tmp/main/scripts/vivadoCreateProject.tcl
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /home/mahdi/logisim_evolution_workspace/HW4-403106681.circ.tmp/main/xdc/vivadoConstraints.xdc
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /home/mahdi/logisim_evolution_workspace/HW4-403106681.circ.tmp/main/scripts/vivadoGenerateBitStream.tcl
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /home/mahdi/logisim_evolution_workspace/HW4-403106681.circ.tmp/main/scripts/vivadoLoadBitStream.tcl

HW4/tb.v /home/mahdi/logisim_evolution_workspace/HW4-403106681.circ.tmp
HW4/tb.v:62: Warning: Calling system function $random() as a task.
HW4/tb.v:62: The functions return value will be ignored.
Normal
Denormal
ACCEPTED
1000 / 1000

```

شکل 14